中图分类号：TN492

学号：ZY2402113

北京航空航天大學

硕士中期检查报告

基于RISC-V的
软硬件协同及编译

作者姓名 蒋一平

学科专业 电子信息

指导教师 李洪革教授，李丽副教授

培养院系 电子信息工程学院

2026年10月

目录

1.开题报告的论文研究方案 1
1.1 研究背景与意义 1
1.2 主要研究内容 1
1.3 关键技术与预期新意 2
2.开题报告的论文工作计划 3
3.已研究的内容和实现技术及关键技术的突破 4
3.1 Linux系统相关 4
3.2 AI编译器相关 4
3.2.1 编译系统总体结构与部署方式 4
3.2.2 模型导入与图中间表示构建 5
3.2.3 硬件约束下的算子分解与执行计划 6
3.2.4 张量物理布局与参数组织 7
3.2.5 静态地址规划与拼接数据布局 8
3.2.6 指令生成、汇编与二进制导出 9
3.2.7 FPGA系统验证与阶段性结果 10
3.2.8 已解决的关键问题与当前限制 11
4.已研究工作总结 11
4.1 Linux系统相关 11
4.2 AI编译器相关 11
5.后续研究的内容和实现技术 12
5.1 面向布局约束的生命周期与存储复用 12
5.2 分支调度与存储布局的联合优化 13
5.3 面向宏指令NPU的后处理融合与能力扩展 14
5.4 数值验证、代价统计与实验设计 14
5.5 后续工作安排与预期成果 16
6.已发表的论文和取得的成果 16
7.参考文献 18

1.开题报告的论文研究方案

1.1 研究背景与意义

近年来，随着嵌入式系统与人工智能应用的发展，端侧设备对计算性能、资源利用效率和系统可定制性的要求不断提高。RISC-V指令集具有开放、模块化和可扩展等特点，为构建自主可控的处理器及片上系统提供了基础。基于FPGA构建集成RISC-V处理器、显示模块和神经网络处理器的SoC，能够为软硬件协同设计提供可重构的实验平台。然而，硬件功能的实现还需要操作系统、设备驱动和编译工具的配套支持。本课题以“基于RISC-V的软硬件协同及编译”为研究主题，围绕Linux系统移植及驱动开发、面向NPU的AI编译器构建与优化两个方面开展研究，以完善自研SoC的软件支持并提升其面向实际应用的运行能力。

在操作系统方面，Linux具有开源、模块化和可移植性较强等特点，能够为应用程序提供进程管理、存储管理和设备访问等基础服务，是完善RISC-V软件环境的重要组成部分。针对FPGA平台资源有限、处理器运行频率受约束以及自研外设缺少配套驱动等问题，需要结合目标SoC设计轻量级引导程序，完成Linux内核、设备树和根文件系统的适配与裁剪，并为图像显示模块、NPU等外设开发相应驱动。通过对启动流程、系统组成与外设访问方式进行协同设计，可以在满足应用功能的基础上控制系统体积和启动开销，为图形交互与异构计算应用提供统一、可靠的软件运行环境。

在编译器方面，神经网络通常以计算图和张量运算描述，而实验室自研NPU采用具有特定编码、数据格式和地址要求的宏指令，两者之间存在模型语义、物理布局和执行控制等方面的差异。AI编译器需要通过计算图解析、算子映射、静态存储规划与指令生成，将模型转换为目标NPU能够执行的程序。已有研究表明，图级优化与算子级映射共同影响模型在目标平台上的执行效率[1]。因此，面向自研NPU构建专用离线编译器，不仅能够建立模型到硬件执行的自动转换流程，还能够结合宏指令粒度、通道对齐及张量依赖等约束，研究执行调度与存储布局的优化方法，为发挥专用加速器的计算能力提供编译支持。

1.2 主要研究内容

本课题的研究内容主要包括以下两个方面。

（1）RISC-VSoC的Linux系统移植及驱动开发。以FPGA实现的RISC-VSoC为基础，研究系统启动、内核适配、根文件系统构建以及外设访问所需的软件支持，建立处理器与片上外设之间的基本软件接口。该部分工作关注操作系统与硬件平台之间的适配关系，为后续系统应用提供支撑。

（2）面向NPU的AI编译器构建与优化。围绕神经网络从模型描述到NPU执行的转换过程，研究模型导入、中间表示、算子合法化、张量布局、参数组织、存储地址规划以及指令生成等关键问题。在完成离线编译与板端部署的基础上，进一步分析宏指令执行方式和多分支网络的数据依赖，探索降低中间张量存储占用及冗余数据搬运的方法。

编译部分采用部署前完成主要决策的技术路线。对于结构和输入形状确定的模型，在PC侧生成固定执行计划、指令序列及参数数据，设备侧负责加载编译产物、配置执行环境、启动NPU并获取结果。TVM等研究说明了将模型级优化与硬件相关代码生成相结合的可行性[2]；本课题根据自研NPU的指令和存储特征，构建范围明确、能够直接服务于原型系统的编译流程。

1.3 关键技术与预期新意

Linux系统方面，关键技术包括面向自研RISC-VSoC的启动环境构建、精简SBI服务实现、Linux内核与根文件系统裁剪，以及设备树适配和定制外设驱动开发。结合开题报告中的研究方案，预期新意在于围绕FPGA平台的资源与频率约束，对引导程序、操作系统配置和外设访问进行协同设计：自主开发仅包含必要硬件初始化、内核加载及启动支持的轻量级引导程序，并结合目标应用裁剪内核与根文件系统，在功能完整性、系统体积和启动效率之间进行权衡；同时，针对图像显示模块、NPU等自研外设建立相应驱动接口，结合硬件条件组织中断响应和数据传输，支撑处理器、显示模块与加速器的协同运行，形成面向本SoC的定制化Linux软件支持方案。

AI编译器方面，关键技术与预期新意主要包括两个方面。一是面向实验室自研、采用宏指令体系的NPU构建专用离线编译器，针对单条指令执行较大粒度张量操作的特点，研究计算图到宏指令执行序列的映射方法，将算子分解、通道分组、参数组织及静态地址规划相结合，在PC侧生成与目标硬件匹配的指令和参数二进制文件，建立从模型描述到RISC-VSoC中NPU执行的编译路径。二是在已有编译流

程基础上，研究面向宏指令及物理布局约束的执行调度与存储联合优化，将张量生命周期、分支依赖、通道对齐和拼接别名关系纳入统一分析，联合选择合法执行顺序与缓冲区复用方案，降低中间张量的峰值存储需求，并减少具备消除条件的数据搬运。前者体现面向自研硬件的专用编译适配，后者作为后续重点研究的优化方法，通过同一硬件平台上的对照与消融实验评价其有效性。

2.开题报告的论文工作计划

开题阶段的总体工作安排按照系统构建、软件适配、编译优化与论文整理依次展开。结合目前已经具备的NPU编译和FPGA验证条件，在保持系统软件与编译两个研究方向的基础上，将编译部分的工作进一步细化为流程完善、约束分析、优化算法实现和板端实验。原计划与当前阶段安排如表1所示。

表1 论文研究工作计划

时间安排	计划内容	阶段说明
2025.12—2026.02	构建RISC-VSoC与FPGA验证环境，集成相关外设	原计划中的硬件平台阶段
2026.03—2026.04	研究启动加载与基础软件适配	系统软件方向
2026.05—2026.07	Linux内核与根文件系统移植	具体进展在Linux章节补充
2026.08—2026.09	外设驱动开发与系统联调	具体进展在Linux章节补充
2026.10—2026.12	面向NPU的AI编译与优化研究	已有全流程基线，继续开展方法与实验
2027.01—2027.04	补充实验、撰写与修改学位论文	优化验证与论文整理交叉进行
2027.05	论文答辩	按学院实际安排执行

目前，AI编译部分已提前形成模型到板端运行的完整验证流程，为原定编译优化阶段提供了可运行的基础。后续工作的重点由功能连通转向研究问题凝练和实验评价，包括建立存储与执行开销统计、实现可独立开关的优化过程，并通过多种网络结构比较优化效果。Linux相关研究的具体阶段进展在相应章节中另行整理，本报告不以编译流程的完成情况替代系统软件工作的完成情况。

为了保证研究工作能够在毕业前完成，后续编译研究以一项明确的存储与调度优化方法为主线，硬件扩展作为具备条件时开展的补充工作。算法设计、正确性验证和性能测量应交叉推进，避免在全部实现完成后才发现评价条件不足。实验阶段将固定模型、输入形状和硬件配置，并记录编译器版本，使各阶段结果能够追溯和复现。

3.已研究的内容和实现技术及关键技术的突破

3.1 Linux系统相关

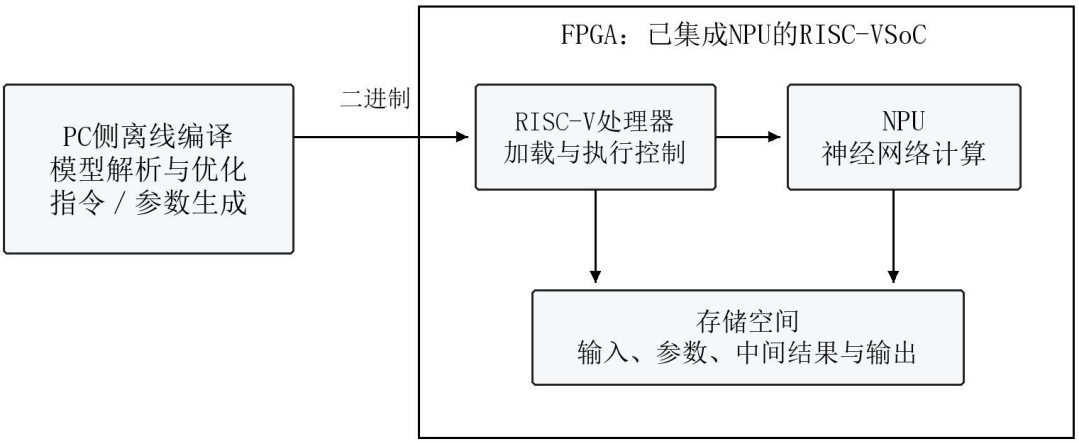
3.2 AI编译器相关

3.2.1 编译系统总体结构与部署方式

目前已构建面向目标NPU的离线AI编译流程，完成从模型导入、图结构解析、存储规划和参数转换，到指令生成、二进制导出及板端运行的衔接。NPU已集成到RISC-VSoC中，整个系统部署至FPGA后，能够运行由本编译器生成的YOLO网络二进制文件，并正确完成图像识别。该结果表明，编译产物已经通过完整SoC环境中的实际执行验证，模型转换、指令编码与硬件执行之间的基本接口能够协同工作。

从工作方式上看，本编译器属于面向静态模型的提前编译系统。计算图解析、算子拆分、执行顺序确定以及张量地址分配均在PC侧进行，端侧主要执行预先生成的指令与数据。这里的“二进制文件”主要指NPU指令镜像和参数镜像，不能直接等同于通用处理器使用的ELF可执行文件。设备侧仍需要完成镜像装载、地址配置和执行启动等工作，离线编译减少了运行时的图分析需求，但不意味着系统不需要加载与控制软件。

图1给出了本课题的系统关系。RISC-V处理器承担控制角色，NPU执行编译器映射的神经网络运算，存储系统提供参数、输入及中间结果空间。编译器与硬件之间的关键契约包括指令格式、地址空间、张量排列及数值表示。实际板端使用的加载接口、图像预处理和结果后处理流程将在最终实验记录中明确，以形成能够重复执行的完整部署说明。



系统结构示意；实际总线与加载接口以平台实现为准

图1 离线AI编译器与RISC-V异构SoC的关系

3.2.2 模型导入与图中间表示构建

模型前端采用PyTorch导出的PT2模型作为输入，通过读取导出程序的计算节点、参数映射和张量元数据，提取后续编译所需的结构化信息。与直接按网络名称生成指令相比，基于图结构的处理方式能够明确算子的输入输出关系，并将模型解析与具体硬件指令生成分离。当前前端与中间表示相关功能分别由模型生成入口、前端转换模块及IR结构定义共同实现。

图中间表示记录算子类别、输入输出张量、形状和算子属性等内容，模型权重与偏置则作为后续参数生成的输入保存。编译器利用静态形状信息确定特征图尺寸及存储需求，随后结合NPU支持范围进行合法化处理。该分层方式与多层次编译基础设施中保留语义、逐级降低的思想一致[3]，但目前实现主要面向本课题需要的算子集合，没有将通用模型框架的全部功能作为支持目标。

当前前端已经建立卷积、激活、平均池化、最大池化、最近邻上采样、填充、通道拼接和全连接等操作到内部表示的映射。需要强调的是，识别到某类算子与支持该算子的全部参数组合并不相同。实际生成目标指令时，还受到通道数量、步长、填充方式、形状及布局等条件限制。对于不满足硬件约束的情况，应在编译阶段明确给出诊断，防止生成表面完整但无法正确执行的产物。

对模型中的部分辅助节点，现有前端采用透传处理以连接有效计算节点。其中，形状整理与数值变换具有不同语义，clamp、类型转换和floor等操作不能在任意模型中被无条件消除。当前已运行模型的端到端验证为相应部署提供了功能依

据，后续仍需结合量化流程和硬件运算定义，对这些处理补充明确的适用条件及逐层数值测试，从而将模型特定经验转化为可检查的转换规则。

3.2.3 硬件约束下的算子分解与执行计划

图中间表示生成后，编译器进一步构建面向NPU的执行计划。该阶段将网络中的逻辑操作转换为硬件能够执行的操作序列，并记录相关参数与存储区域之间的对应关系。执行计划是图层与指令层之间的重要中间环节：图层保留网络连接关系，执行计划描述具体如何分组执行，指令层再将相关信息编码为目标格式。其总体流程如图2所示。

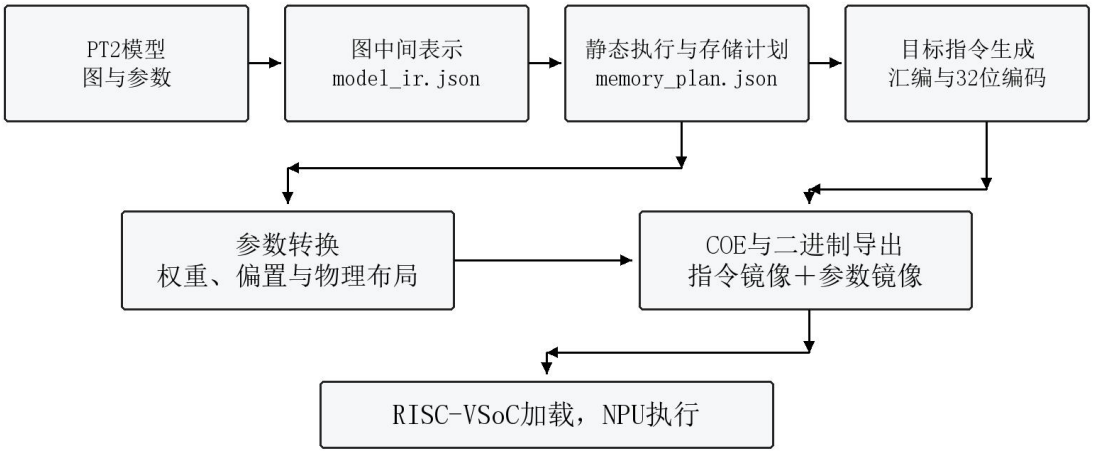


图2 已实现的AI编译流程与主要中间产物

卷积是YOLO网络中的主要计算操作之一。当前编译器根据目标硬件限制对输出通道进行分组，并使用4通道或8通道分组策略组织部分执行过程。分组选择与特征图规模阈值等配置有关，逻辑输出通道较多时会被拆分为多次硬件操作。该实现解决了模型通道数与单次指令处理能力之间的映射问题，同时要求输入地址、输出区域、权重排列和偏置组织保持一致。

对于部分无法直接按原始形式映射的操作，编译器使用已有硬件能力构造执行序列。例如，某些卷积配置会在计划中进一步结合下采样处理，特定填充与池化模式具有专门的组合处理路径。此类映射属于针对现有硬件语义的合法化，不能仅依据算子名称推断其与任意标准框架配置等价。后续验证将重点覆盖这些组合路径的边界尺寸和数值行为。

在激活相关处理上，现有执行计划能够关联卷积与后续激活，但部分路径仍生成独立卷积和激活指令，并保留相应中间存储。因此，目前不能将其统一表述为消

除了中间访存的硬件融合。对于全连接等操作，编译器也根据目标指令的计算粒度生成相应序列。阶段性工作已经完成从高层算子到合法目标操作的映射，后续将进一步分析分组策略及指令组合是否存在优化空间。

3.2.4 张量物理布局与参数组织

目标NPU对输入、输出和参数采用特定的数据排列方式，框架中的逻辑张量不能直接作为硬件存储镜像使用。当前实现按照4通道对齐等规则组织特征图和参数，并在必要时补齐不满足对齐要求的通道。布局变换使数据能够匹配硬件的读取方式，但也会引入填充空间。因此，存储分析应以真实物理字节数为依据，而不能只用原始张量元素数量估计占用。

以通道数不是4的整数倍的特征图为例，逻辑上的有效元素数与实际分配空间可能不同。地址计算需要同时考虑空间位置、通道组以及组内通道偏移，且生产者与消费者必须使用相同的解释。若某一阶段按照逻辑通道数计算地址，另一阶段按照对齐后的通道数访问，就可能产生跨层数据错误。编译器通过统一计划中的布局信息连接参数生成与指令生成，减少上述不一致。

参数生成阶段依据执行计划处理卷积权重和偏置，并输出符合硬件读取顺序的数据。具有偏置与不具有偏置的操作需要对应不同的有效参数内容，不能简单假设每个卷积均附带相同长度的偏置区。参数镜像与指令中引用的地址必须一一对应，这也是编译流程中需要保持跨阶段一致性的关键问题。

图3以示意形式说明通道对齐与存储规划之间的关系。图中的填充空间用于展示布局约束，不代表当前某一层模型的实测占用。后续实验将分别统计有效数据、对齐填充和地址间隙，以区分模型本身需求与编译布局带来的额外开销，为优化策略提供依据。

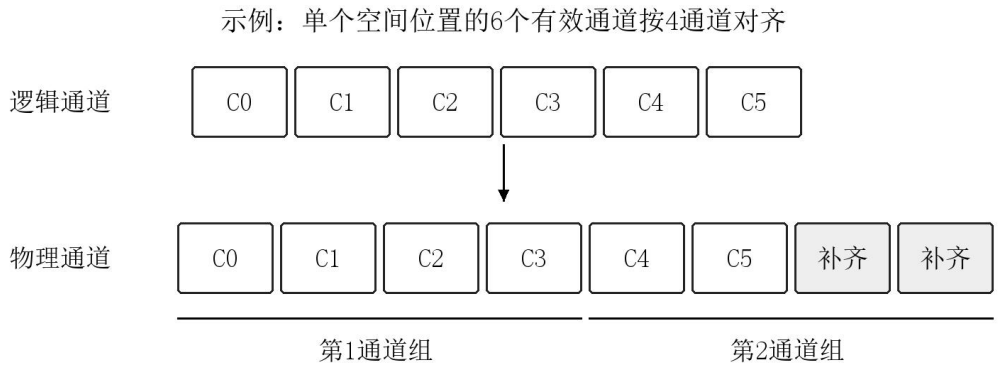


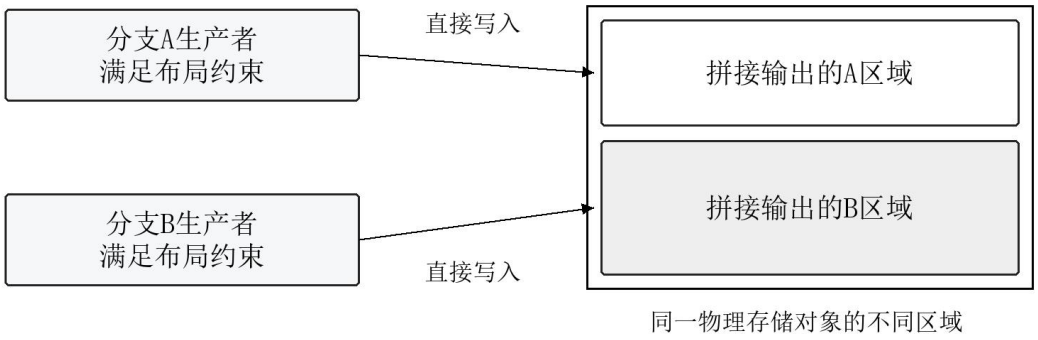
图3 通道对齐对物理存储空间的影响示意

3.2.5 静态地址规划与拼接数据布局

当前编译器在PC侧完成参数区、运行区和输出区的地址规划，生成包含张量区域、参数位置及执行信息的存储计划。后续参数导出和指令生成统一读取该计划，因此无需在设备运行过程中重新推导整张计算图的地址。静态地址规划适合本课题输入形状确定、硬件地址空间明确的部署方式，有利于降低端侧执行组织的复杂度。

现有运行区分配主要采用按需求向后扩展的顺序分配方式，便于保证不同中间结果在地址上的独立性，但尚未形成基于全图生命周期的通用缓冲复用机制。当网络层数或分支数量增加时，已经完成全部消费的张量仍可能占用原先分配的空间，使工作区需求随中间结果累积。因此，当前存储计划既是完整编译流程的重要组成部分，也是后续优化的主要切入点。

对于满足条件的通道拼接，现有编译器已采用生产者输出预布局方法，将各分支结果写入拼接输出对应的区域，避免再生成独立的数据拷贝。该方法具有明确适用条件，包括沿通道维拼接、空间尺寸一致以及通道对齐满足目标布局规则等。图4给出了这一处理的基本关系。它说明现有编译器并非只进行机械指令翻译，而是已经利用部分图结构信息改善数据组织。



无额外拼接拷贝；复用时必须共同检查别名与消费者

图4 满足布局条件时的通道拼接免拷贝示意

拼接预布局也使后续存储复用更加复杂。不同逻辑张量可能对应同一物理存储对象中的不同区域，若分别计算并释放其地址，就可能破坏仍被后续算子使用的数据。因此，后续方法需要将别名及子区域关系纳入统一分析，使缓冲区复用能够与既有免拷贝布局共同工作，而不是在引入复用后破坏当前已经实现的优化。

3.2.6 指令生成、汇编与二进制导出

在执行计划和存储计划确定后，后端根据算子类型调用相应指令生成模板，将输入输出地址、参数地址以及操作配置转换为汇编形式的指令序列。汇编模块进一步按照目标编码规则生成32位指令字，并由后续脚本转换为初始化文件和二进制镜像。完整流程由构建脚本组织，减少了手工运行多个转换步骤所带来的产物不一致。

现有指令体系包含配置指令以及卷积、池化、上采样、全连接等计算相关指令。部分运算需要先发送多条配置指令，再通过计算指令启动硬件执行。由于一条计算指令通常对应一类较大粒度的张量操作，该体系具有宏指令特征。宏指令简化了编译器对内部计算循环的控制需求，但也意味着许多细粒度实现选择由硬件固定，软件优化需要集中在硬件实际暴露的能力上。

构建过程能够输出用于FPGA存储初始化的COE文件，并进一步生成npu_instr.bin与npu_params.bin等二进制文件。其中，指令和参数具有不同的用途及装载位置，应结合存储计划使用。当前导出流程采用小端字节序组织指令数据，并通过地址范围检查等逻辑约束产物。此类二进制文件已经在上述SoC与FPGA验证环境中用于YOLO网络执行。

需要区分的是，指令编码器支持某种操作码，并不等于模型前端已经支持所有包含该运算的网络。模型导入、合法化、参数生成、指令生成与数值验证均完成后，才能形成完整算子支持。后续将整理算子能力表，分别列出前端识别范围、后端合法参数范围及已验证案例，使编译器的支持边界更加清晰。

3.2.7 FPGA系统验证与阶段性结果

本阶段已经完成NPU在RISC-VSoC中的集成验证，并将整个系统部署到FPGA。板端能够加载本编译器生成的YOLO网络二进制产物并正确进行图像识别，说明现有流程已经贯通模型转换、参数组织、指令生成、文件装载及硬件执行。该成果是后续研究的实验基础，可以用于比较不同编译策略在同一硬件平台上的行为。

当前确认的结果主要属于端到端功能验证。尚未整理成完整实验记录的内容包括FPGA型号、SoC与NPU工作频率、所用YOLO版本和输入尺寸、量化配置、测试样本范围以及端到端执行时间等。因此，本阶段不据此给出帧率、功耗、加速比或数据集精度等量化结论，也不将可观察的识别成功等同于逐层结果完全一致。需要补充的实验记录如表2所示。

表2 FPGA验证记录与待补充实验信息

项目	当前状态	需归档的证据
系统集成与运行	已完成RISC-VSoC集成NPU及FPGA运行	系统版本、板卡照片、运行日志
YOLO图像识别	已运行编译器生成的二进制并正确识别	真实输入与输出图像、模型及样本说明
硬件与模型配置	待补充完整记录	FPGA型号、频率、模型版本、输入尺寸、量化格式
数值一致性	待补充系统性测试结果	软件参考、逐层输出、误差指标
存储与性能数据	待测量并汇总	工作区峰值、执行周期、计时边界及重复次数

图5待插入：RISC-VSoC的FPGA实物与YOLO运行结果。建议左侧放置FPGA开发板实物照片，标注板卡型号及图像输入、结果输出接口；右侧放置同次运行的输入图像与真实识别结果，并附模型名称、输入尺寸和测试日期。下方补充装载npu_instr.bin、npu_params.bin及执行完成的真实日志。不得以流程示意图代替实测证据。

图5 FPGA系统运行与YOLO图像识别结果（实测材料待补充）

代码中已经包含二进制导出、地址限制、重建一致性及部分模型回归等检查，为后续开发提供了基础。但这些检查主要关注文件和编译流程的一致性，不能替代硬件数值验证。后续将以已运行模型为基线，在补充单算子参考结果和中间张量比对后，再评估优化版本，确保性能变化建立在功能正确的前提上。

3.2.8 已解决的关键问题与当前限制

本阶段解决的主要问题是将模型图、物理存储布局与NPU指令连接为可运行的完整流程。通过结构化图表示与执行计划，编译器能够处理模型依赖并将较大的逻辑算子拆分为硬件支持的操作；通过统一存储计划，参数与指令能够引用一致的地址；通过COE和二进制导出，编译结果能够进入实际SoC环境执行。上述工作已由FPGA上的YOLO运行验证支撑。

目前仍存在三个方面的限制。首先，运行区以顺序分配为主，尚未充分利用张量生命周期进行空间复用。其次，部分布局与分组策略依赖固定规则，缺少基于代价模型的系统选择。最后，现阶段的验证重点是功能连通，尚需补充能够支撑优化结论的统一测量与数值比对方法。这些限制构成后续研究的具体问题，不影响已完成全流程功能验证这一阶段性结论。

4.已研究工作总结

4.1 Linux系统相关

4.2 AI编译器相关

按照编译部分的阶段目标，目前已经完成从PT2模型导入到NPU二进制文件生成的主要软件流程，形成图表示、执行计划、存储计划、参数转换和指令生成等模块，并完成在RISC-VSoC集成环境下的FPGA部署验证。编译器生成的YOLO二进

制文件已经能够在该系统上运行并正确进行图像识别，表明模型到硬件执行的端到端路径已经跑通。

与开题阶段的总体方案相比，目前编译研究已结合具体NPU实现进一步明确了目标平台、输入模型和输出形式，将重点落实到静态推理的图分析、数据布局和专用指令生成。研究主线仍然是RISC-V平台上的软硬件协同与编译，后续需要在现有工程基础上继续提炼优化方法。当前完成的流程建设提供了验证环境，但尚不能单独作为编译性能优于已有方法的依据。

结合文献分析，MCU部署研究已经表明全图存储管理和资源约束建模的重要性，DORY与MCUNet分别从分块规划和网络—执行引擎协同角度提出了相关方法[4][5]。本课题的进一步研究应针对宏指令NPU的布局和执行约束，明确现有通用方法直接应用时遇到的问题，并通过算法设计与对照实验体现改进价值。下一阶段将优先围绕存储复用、分支调度和拼接别名的共同约束开展研究。

5.后续研究的内容和实现技术

5.1 面向布局约束的生命周期与存储复用

后续首先研究静态图中张量的产生、使用和释放关系，在现有存储计划生成之前增加生命周期分析。对每个中间张量记录生产节点、全部消费者、物理尺寸及对齐要求，并在确定执行顺序后计算有效存活区间。对于已经没有后续使用的张量，其空间可以提供给后续结果复用。外部输入、模型输出、长期参数与临时激活应分别管理，避免将需要保留的数据错误回收。

基础实现将采用按执行顺序释放空间并选择可容纳区域的静态分配方法，同时保留现有顺序分配作为对照。随后针对本平台的4通道布局与拼接预布局建立物理存储对象，将共享区域的逻辑张量作为视图管理。在保守实现中，只有全部相关视图完成消费后才允许回收底层空间；进一步的子区域复用则需要证明各读写范围不重叠，并建立相应合法性检查。

研究重点不只是添加一个通用内存池，而是处理布局、别名和目标指令访问范围共同造成的约束。例如，某个分支张量已被直接消费者使用完毕，但其存储仍属于尚未完成消费的拼接输出，此时不能仅依据该分支的逻辑生命周期释放空间。拟

通过统一的存储对象描述将此类约束显式化，并比较忽略别名的简单方法、保守别名方法与联合规划方法在正确性及占用方面的差异。

评价指标包括峰值运行区大小、对齐填充量、空间碎片以及编译耗时。峰值存储需求下降并不必然带来推理加速，因此将存储收益与时间收益分别报告。若存储复用使某些原本超出地址预算的网络能够部署，也将以固定预算下的可部署性作为独立结果，而不使用未经测量的性能推断。

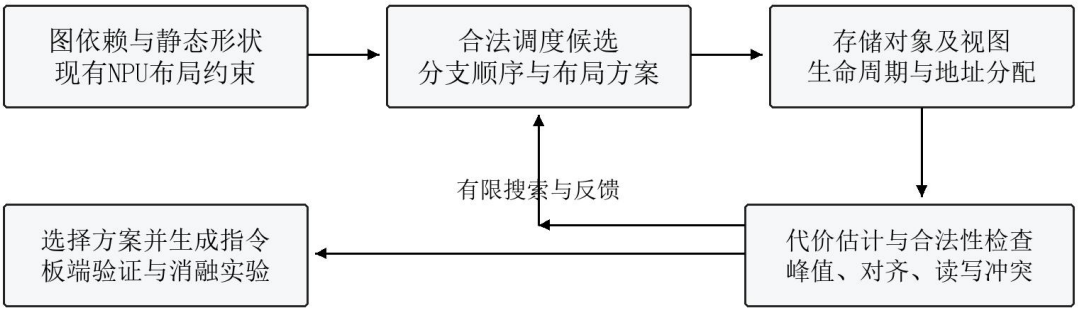
5.2 分支调度与存储布局的联合优化

具有多分支和拼接结构的网络，在满足依赖关系的前提下可能存在多种合法执行顺序。不同顺序会改变共享输入与分支输出的保留时间，从而影响峰值存储。MAGIS等研究已说明图变换与执行调度之间存在共同优化空间[6]。针对本课题，拟在既有拼接免拷贝处理基础上，将合法拓扑顺序、张量生命周期和物理布局共同纳入决策，而不是固定顺序后仅分配地址。

具体实现首先从当前可执行节点集合中选择候选操作，估计执行后新产生的数据、能够释放的数据及对后续汇合节点的影响，形成面向峰值占用的启发式评分。对于规模较小的分支子图，可以采用有限宽度搜索或枚举获得对照结果；对于完整网络，则限制候选数量和搜索时间，避免编译开销过大。每个候选方案均需通过拓扑合法性和存储访问冲突检查。

联合优化还需处理局部最优与全局效果之间的关系。先完成某一短分支可能暂时释放部分输入，但其输出需要等待另一长分支，从而延长存活时间；反之，优先执行长分支也可能增加中间峰值。因此，候选评分除当前释放量外，还应考虑汇合距离和输出保留需求。研究将通过典型分支子图解释算法行为，并通过完整YOLO网络验证其实际影响。

拟设置固定顺序加顺序分配、固定顺序加生命周期复用、仅调整分支顺序以及联合调度与布局规划等对照。通过消融实验分析收益分别来自空间复用、顺序改变还是布局兼容处理，避免把多个机制合并后仅给出总结果。该方向预期形成可独立描述的算法、合法性条件和实验结论，是后续论文工作的主要研究内容。



后续拟实现方法；当前已完成基线编译与FPGA功能验证

图6 后续联合调度与存储规划方法的技术路线

5.3 面向宏指令NPU的后处理融合与能力扩展

在完成软件优化基线后，将依据指令统计和访存分析判断是否需要扩展NPU能力。DNNFusion等工作表明融合可以减少中间结果和执行开销，但其有效性取决于目标硬件的实现方式[7]。本课题优先考虑出现频繁且语义明确的模式，例如在卷积输出阶段结合激活处理，以减少单独激活指令及中间结果的读写。是否实施该扩展将由实际瓶颈和硬件修改成本决定。

实现时需要与硬件设计协同确定激活模式、输出位宽、量化移位及饱和顺序等语义，并在指令中提供明确配置。编译器根据子图模式和数值条件选择融合路径，不满足条件时保留原有执行方式。若一个卷积输出同时被多个消费者使用，或者某个消费者需要激活前数据，则不能直接用融合结果替代全部用途，必须分析依赖与数据保留要求。

硬件比较将同时记录资源占用、最高可工作频率及推理周期，区分指令减少、访存减少和时钟变化带来的影响。DeFiNES的分析说明，跨层执行需要同时考虑不同存储层次的访问成本[8]。在本平台缓存容量、片上缓冲形式和DMA能力确认之前，不预设双缓冲或传输计算重叠一定可用；这些能力只在接口条件明确后作为补充方案进行评估。

5.4 数值验证、代价统计与实验设计

正确性验证将从现有YOLO端到端结果扩展到分层验证。首先整理硬件整数或定点运算约定，建立与权重、偏置、累加、移位和饱和行为一致的软件参考。量化推理研究表明，数值格式与量化位置是整数计算正确执行的重要组成部分[9]。在此

基础上，对卷积、池化、激活及典型组合子图逐层比对，再验证完整网络的输出与任务表现。

测试用例将覆盖奇数空间尺寸、非4倍数通道、多消费者张量、连续拼接以及数值边界等情况。对于仅改变地址与执行顺序的优化，原则上要求与同一硬件基线保持相同结果；对于改变数值计算顺序的融合，需预先定义可接受误差及任务指标约束。正确性检查与性能测试使用相同的模型配置，并记录编译产物和硬件版本，减少版本变化对结论的干扰。

静态分析方面，将从存储计划和指令序列统计工作区峰值、参数大小、指令条数及可计算的数据读写量。动态测量方面，将分别记录NPU执行周期和包含加载、预处理、后处理的端到端时间，明确计时边界。若硬件提供周期计数或事件计数接口，优先使用该接口；否则使用可校准的板端计时方式。所有实验保持时钟、输入和测量流程一致，并报告重复测量次数及波动范围。

代表性负载拟包含单链卷积、分支拼接子图以及当前已运行的YOLO网络，在兼容范围内改变输入尺寸和通道配置，以检查方法对不同图结构的适用性。跨编译器比较仅在目标硬件和算子支持具有可比性时开展，主要采用同一后端上的优化开关进行对照。表3列出了实验组与主要评价内容。

表3 编译优化对照实验设计

实验组	配置	主要比较目的
基线组	固定顺序＋现有顺序分配	建立功能、存储与周期基线
生命周期组	固定顺序＋别名安全的空间复用	分析静态复用带来的容量收益
调度组	改变合法分支顺序，保持统一分配策略	分析调度对生命周期和峰值的影响
联合优化组	分支调度＋布局约束＋空间复用	评价联合决策相对独立阶段的收益
融合扩展组（条件性）	加入经验证的硬件后处理融合	比较访存、周期、频率与硬件资源
共同验证	相同模型、输入、频率和数值约定	逐层正确性、任务表现与可复现性

5.5 后续工作安排与预期成果

后续研究按照“基线记录—方法实现—对照验证—论文整理”的顺序推进。首先补齐硬件能力说明、YOLO配置及板端证据，建立可重复执行的测试流程；其次实现生命周期分析与别名约束下的静态复用，形成稳定的存储优化基线；随后开展分支调度与布局联合优化，并进行消融实验；最后根据测量结果决定是否加入小范围硬件扩展，完成论文分析与材料整理。具体安排如表4所示。

表4 中期检查后的研究安排

时间	主要任务	阶段产物
2026.10—2026.11	整理硬件约束与基线；实现生命周期和别名分析	能力说明、数值测试、存储分析
2026.12—2027.01	实现布局约束下的复用及分支联合调度	算法原型、合法性检查、对照结果
2027.02—2027.03	完成多负载测试及消融；按条件评估硬件扩展	FPGA实测数据与适用范围分析
2027.03—2027.04	整理实验，完成论文写作与修改	论文初稿、修改稿与答辩材料
2027.05	完成答辩及材料归档	按学院安排完成提交

预期形成的成果包括一套能够稳定生成目标NPU指令与参数的离线编译工具链、一项面向通道布局与别名约束的联合调度和存储优化方法，以及相应的正确性测试和FPGA实验结果。若硬件扩展验证能够在既定时间内完成，则补充编译映射与硬件代价分析。论文将以实际实验结果为依据说明收益及适用范围，不预设未经验证的加速比例或存储降低幅度。

6.已发表的论文和取得的成果

截至本次中期检查，尚无与本课题相关的已发表论文、专利或软件著作权。

目前已经取得的阶段性工程成果主要包括：完成面向目标NPU的模型解析、图表示、执行计划和存储规划；实现参数转换、指令生成与二进制导出；形成贯通PC侧编译与FPGA侧执行的部署流程。NPU已集成至RISC-VSoC并部署在FPGA

上，能够运行编译器生成的YOLO二进制文件并正确完成图像识别。这些成果为后续编译优化提供了软件与实验平台基础。

现阶段成果与开题研究方向中的AI编译及软硬件协同部分相对应。后续将围绕联合存储与调度优化整理可复现的算法说明、测试用例和实验数据，在形成充分验证后进一步凝练论文贡献。工程流程完成情况与尚待研究的优化方法在报告中分别说明，避免将计划中的成果提前列为已取得成果。

7.参考文献

[1] Li M, Liu Y, Liu X, et al. The Deep Learning Compiler: A Comprehensive Survey[J]. IEEE Transactions on Parallel and Distributed Systems, 2021, 32(3): 708-727. DOI: 10.1109/TPDS.2020.3030548.

[2] Chen T, Moreau T, Jiang Z, et al. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning[A]. In: 13th USENIX Symposium on Operating Systems Design and Implementation[C]. Berkeley: USENIX Association, 2018: 578-594. https://www.usenix.org/conference/osdi18/presentation/chen.

[3] Lattner C, Amini M, Bondhugula U, et al. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation[A]. In: 2021 IEEE/ACM International Symposium on Code Generation and Optimization[C]. Piscataway: IEEE, 2021: 2-14. DOI: 10.1109/CGO51591.2021.9370308.

[4] Burrello A, Garofalo A, Bruschi N, et al. DORY: Automatic End-to-End Deployment of Real-World DNNs on Low-Cost IoT MCUs[J]. IEEE Transactions on Computers, 2021, 70(8): 1253-1268. DOI: 10.1109/TC.2021.3066883.

[5] Lin J, Chen W M, Lin Y, et al. MCUNet: Tiny Deep Learning on IoT Devices[A]. In: Advances in Neural Information Processing Systems 33[C]. Red Hook: Curran Associates, 2020: 11711-11722.

[6] Chen R, Ding Z, Zheng S, et al. MAGIS: Memory Optimization via Coordinated Graph Transformation and Scheduling for DNN[A]. In: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3[C]. New York: ACM, 2024: 607-621. DOI: 10.1145/3620666.3651330.

[7] Niu W, Guan J, Wang Y, et al. DNNFusion: Accelerating Deep Neural Networks Execution with Advanced Operator Fusion[A]. In: Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation[C]. New York: ACM, 2021: 883-898. DOI: 10.1145/3453483.3454083.

[8] Mei L, Goetschalckx K, Symons A, et al. DeFiNES: Enabling Fast Exploration of the Depth-First Scheduling Space for DNN Accelerators through Analytical Modeling[A]. In: 2023 IEEE International Symposium on High-Performance Computer Architecture[C]. Piscataway: IEEE, 2023: 570-583. DOI: 10.1109/HPCA56546.2023.10071098.

[9] Jacob B, Kligys S, Chen B, et al. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference[A]. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition[C]. Piscataway: IEEE, 2018: 2704-2713. DOI: 10.1109/CVPR.2018.00286.